

How To Train Lora / Fine-Tune Anima Base Model

Anima Portable Standalone Trainer — Part 2: How to Train a LoRA

This is **Part 2** of a **3-part** series.

- [Part 1: Installation](#)
- **Part 2: How to Train a LoRA** ← You are here
- [Part 3: How to Finetune a Model](#)

⚠ **Resource Notice:** Due to recent updates on the CivitAI platform, some file attachments in this article may no longer be visible or accessible. For all resources related to this article — including additional tools, files, and miscellaneous assets — please refer [HERE](#)

Prerequisites

Before continuing, make sure you've completed [Part 1](#) and have:

- The trainer fully installed and the Web UI launching at <http://localhost:3000>
- All three model weights downloaded into the `models/` folder
- Node.js v18+ installed

📁 Step 1 — Prepare Your Dataset

A good dataset is the most important part of LoRA training. Quality matters far more than quantity, but you still need enough images to learn from.

Minimum recommended: 50 images or 100 images

Caption Your Images

Every image needs a corresponding `.txt` file with the same filename containing its caption. The recommended format is a **trigger word** followed by a descriptive natural language sentence:

```
Kurusaki, A girl with red and yellow heterochromia eyes, black uneven twintails, She is ...
```

💡 **Tip:** For best results with natural language / paragraph-style captions, use [JoyCaption](#) — it's specifically tuned for this kind of descriptive captioning.

Trigger-Word adder can be downloaded from [Here](#)

Caption rules:

- Always lead with your **trigger word** (e.g. `Kurusaki`). This is the word you'll use at inference time to activate the LoRA.
- Follow it with a descriptive sentence covering the subject's key visual features.
- Keep captions consistent in across your dataset.
- Save captions as `.txt` files alongside their matching images.

Organize Your Folder

Put all your images and their `.txt` captions into a single folder, for example:

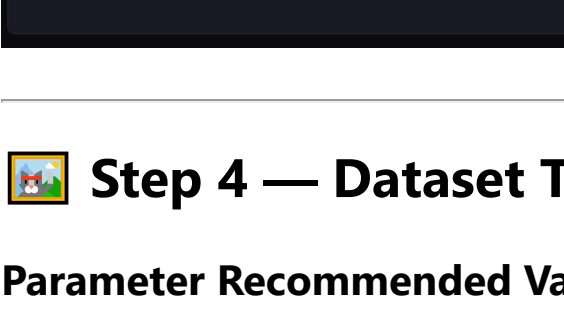
```
C:\dataset\KuruSaki\  
├─ image_001.jpg  
├─ image_001.txt  
├─ image_002.jpg  
├─ image_002.txt  
└─ ...
```

NEW Step 2 — Create a New Job

Launch the Web UI (`training-ui\start_windows.bat`) and open <http://localhost:3000> in your browser.

Click + **New Job** to create a new training job. Give it a name — this becomes your output filename (e.g. `KuruSaki`).

The job editor opens with several tabs across the top: **Training**, **Dataset**, **Network**, **GPU**, **Prompts**, **Samples**, **Console**, **TensorBoard**. We'll go through each relevant tab below.



⚙ Step 3 — Training Tab

The **Training** tab is where you set model paths and core hyperparameters.

Anima Model Paths

At the top of the Training tab, verify that all three model paths are correctly pointing to your downloaded weights:

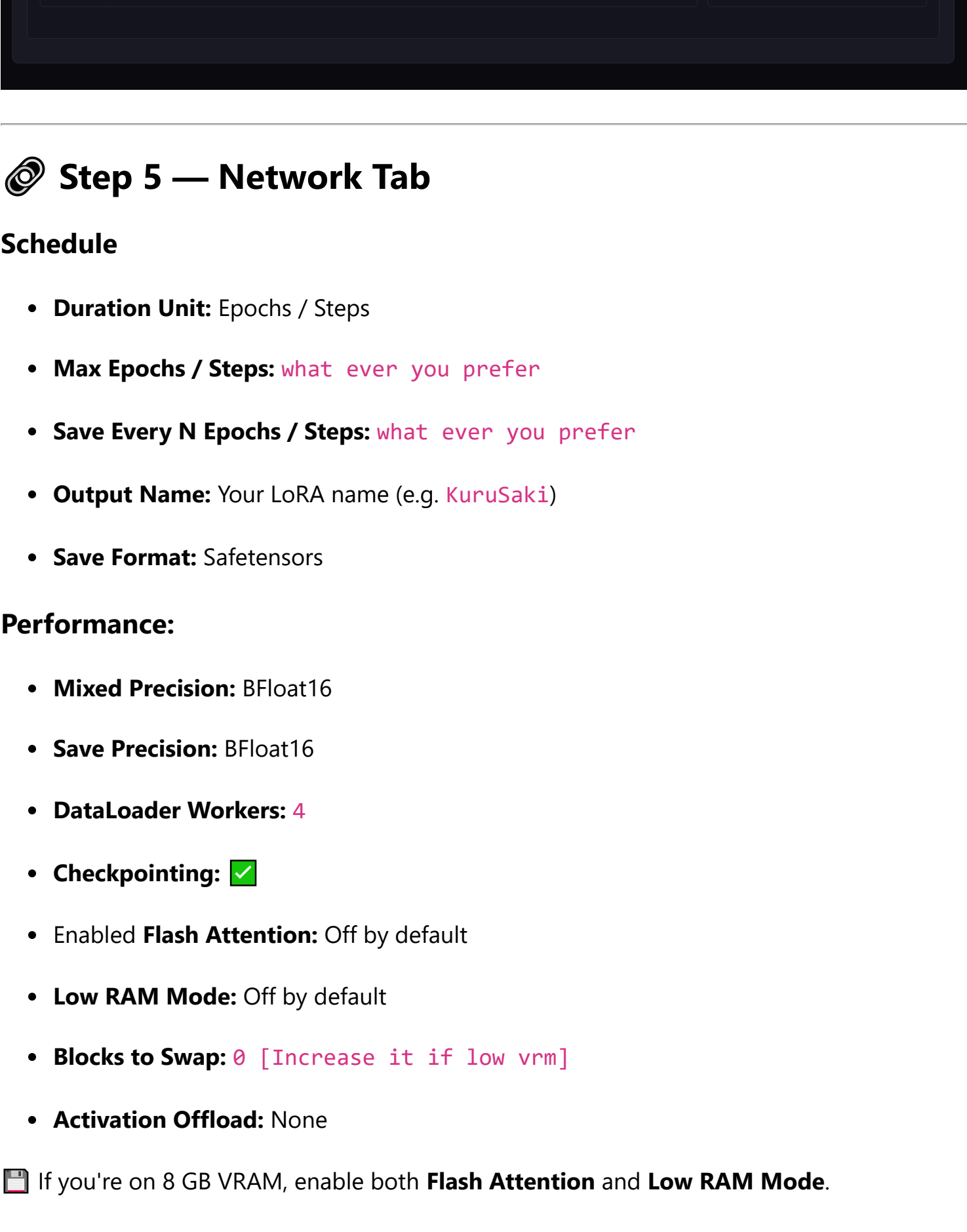
- Field Path DiT Model Path `...\Models\anima_baseV10.safetensors`
- Qwen3 Text Encoder Path `...\Models\qwen_3_06b_base.safetensors`
- VAE Path `...\Models\qwen_image_vae.safetensors`

📘 These paths are globally shared — updating them here updates them for all jobs.

Optimization Settings

Parameter Recommended Value

- Learning Rate `0.0001` or `0.00001`
- Text Encoder LR `0.00005`
- Optimizer `AdamW8bit`
- LR Scheduler `Cosine`
- LR Warmup Steps `0`
- Weight Decay `0.01`
- Seed `42`



🖼 Step 4 — Dataset Tab

Parameter Recommended Value

- Resolution `1536` (*max resolution for bucketing*)
- Batch Size `1` [**Increase it if you have high vram**]
- Gradient Accumulation `1`
- Caption Extension `.txt`
- Alpha Mask Loss: Off

Bucketing

Parameter Recommended Value

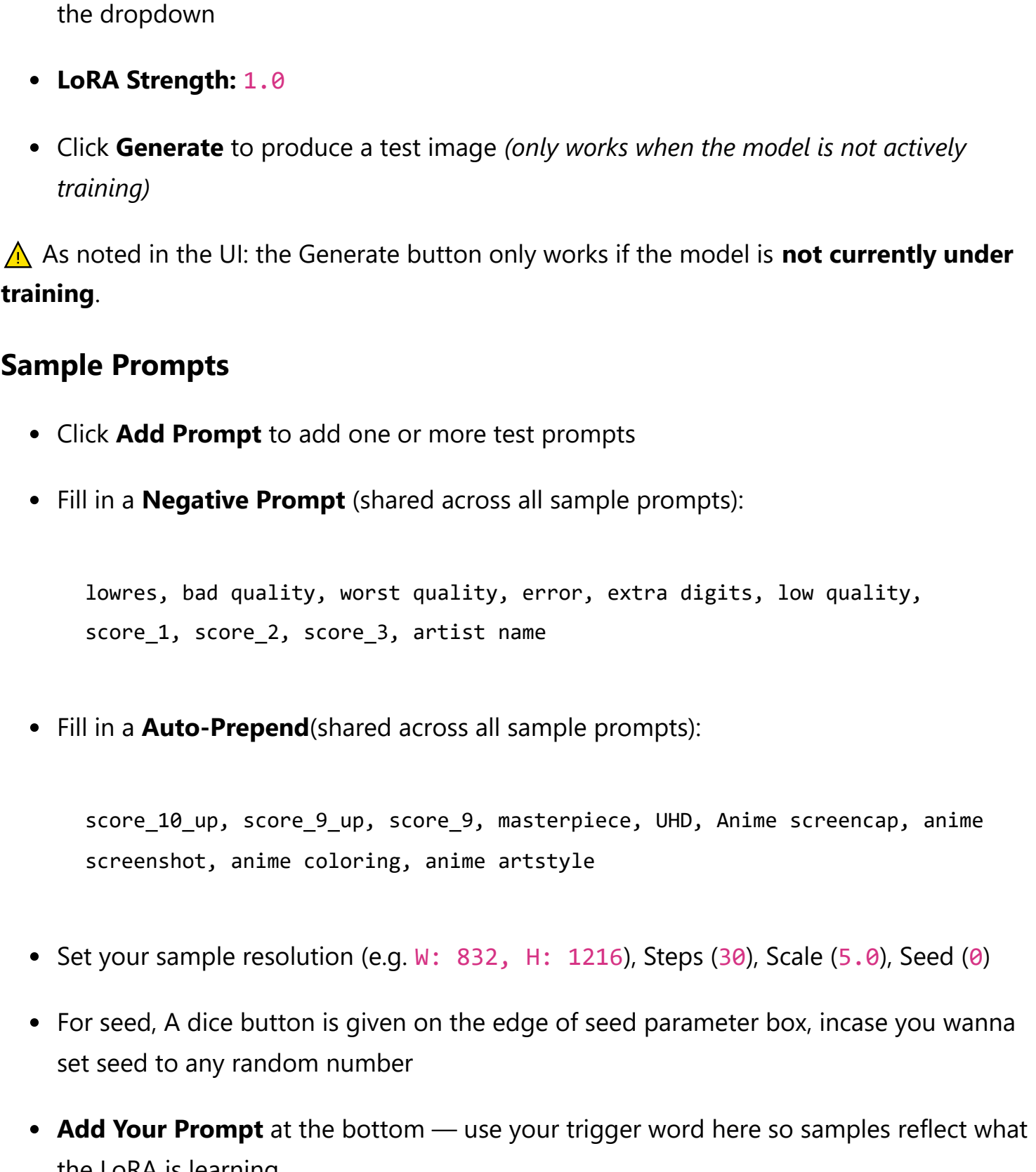
- Enable Aspect Ratio Bucketing ☒
- Enabled Do Not Upscale ☒
- Enabled Min Bucket Resolution `512`
- Max Bucket Resolution `1536`
- Bucket Steps `64`

⚠ Aspect ratio bucketing allows training on images of mixed sizes without cropping or distorting them. Always keep **Do Not Upscale** checked — upscaling low-res images degrades quality.

Dataset Folders

Under **Dataset Folders**, click **Add Folder** and fill in:

- **Image Folder Path:** the path to your captioned image folder (e.g. `C:\dataset\concept`)
- **Repeats:** `1` if you have 300 images; `3` if you have ~100 images (to reach an equivalent dataset size) adjust this according to your dataset, I have just given an example here as i use between 100-300.



🌀 Step 5 — Network Tab

Schedule

- **Duration Unit:** Epochs / Steps
- **Max Epochs / Steps:** **what ever you prefer**
- **Save Every N Epochs / Steps:** **what ever you prefer**
- **Output Name:** Your LoRA name (e.g. `KuruSaki`)
- **Save Format:** Safetensors

Performance:

- **Mixed Precision:** BFloat16
- **Save Precision:** BFloat16
- **DataLoader Workers:** `4`
- **Checkpointing:** ☒
- Enabled **Flash Attention:** Off by default
- **Low RAM Mode:** Off by default
- **Blocks to Swap:** `0` [**Increase it if low vrm**]
- **Activation Offload:** None

📘 If you're on 8 GB VRAM, enable both **Flash Attention** and **Low RAM Mode**.

Caching:

- **Cache Latents to Disk:** ☒
- Enabled **Cache Text Encoder Outputs to Disk:** ☒
- Enabled **Timestep Method:** Logit Normal
- **Flow Shift:** `3`



🖥 Step 6 — GPU Tab

Your available GPU(s) will be listed under **GPU Selection**. Click to select the GPU you want to use (e.g. `GPU 0 | 16303 MiB GB`).

Under **Multi-GPU Mode**, leave it set to **DDP — Data Parallel (standard)** unless you are running multiple GPUs, in which case FSDP or DeepSpeed can reduce VRAM by sharding the model.

🤖 Step 7 — Prompts Tab

The **Prompts** tab lets you configure **in-training image sampling** — the UI will automatically generate test images at your chosen interval so you can visually track training progress without stopping the job.

In-Training Sampling

- **Enable Sampling During Training:** ☒ Enabled
- **Sample Every N Epochs / Steps :** `5` or **what ever you prefer**

Test Generation

This section lets you test your LoRA checkpoint manually at any point:

- **LoRA Checkpoint:** Select the version of the LoRA or checkpoint you want to test from the dropdown
- **LoRA Strength:** `1.0`
- Click **Generate** to produce a test image (*only works when the model is not actively training*)

⚠ As noted in the UI: the Generate button only works if the model is **not currently under training**.

Sample Prompts

- Click **Add Prompt** to add one or more test prompts
- Fill in a **Negative Prompt** (shared across all sample prompts):
`lowres, bad quality, worst quality, error, extra digits, low quality, score_1, score_2, score_3, artist name`
- Fill in a **Auto-Prepend**(shared across all sample prompts):
`score_10_up, score_9_up, score_9, masterpiece, UHD, Anime screencap, anime screenshot, anime coloring, anime artstyle`
- Set your sample resolution (e.g. **W:** `832`, **H:** `1216`), Steps (`30`), Scale (`5.0`), Seed (`0`)
- For seed, A dice button is given on the edge of seed parameter box, incase you wanna set seed to any random number
- **Add Your Prompt** at the bottom — use your trigger word here so samples reflect what the LoRA is learning

📊 Step 8 — Monitor with TensorBoard

This opens TensorBoard in a new browser tab (takes few seconds), showing your training loss curves in real time. The UI automatically finds a free port and connects — no manual setup needed. Keep an eye on your loss curve: it should steadily decrease and begin to plateau. If it's not decreasing at all, your learning rate may be too low. If it's spiking wildly, it may be too high.

💾 Step 9 — Save and Start Training

Once all tabs are configured, click the **Save** button in the top right to save your job configuration.

Then hit **Start Training**. The **Console** tab will show live training output, and the **Samples** tab will populate with generated images at your chosen epoch interval.

✅ Training Complete

When training finishes, your LoRA files will be saved to the output folder in `.safetensors` format, named after your job (e.g. `KuruSaki_epoch20.safetensors`). You can load these directly into any Anima-compatible inference pipeline or ComfyUI workflow.

Note: If you're using RunPod, do use my [referral link!](#) ❤